

Dr. D. Y. Patil Educational Federation's
Dr. D. Y. PATIL COLLEGE OF ENGINEERING & INNOVATION
Survey No. 27/A/1/2C, Varale Campus, Near Talegaon Railway Station,
Tal. Maval, Dist. Pune 410 507, Ph: 020 48522561, 565,566
Web Site: www.dypcoei.edu.in



Subject: Laboratory Practice VI (410256)
Natural Language Processing Laboratory
Course: - 2019

Assignment No. 04

Name of the Student:_____ **Roll no:**_____

CLASS: - B.E. [Computer]

Division: B/C

Marks: ____/10

Date of Checking: ____/____/____

Sign with Date: _____

ASSIGNMENT NO: 04**TITLE:**

To study a transformer from scratch using python.

AIM:

Create a transformer from scratch using the Pytorch library

Theory:

Building a simple Google Translate-like machine translation model using PyTorch involves creating a sequence-to-sequence (Seq2Seq) architecture, with an Encoder-Decoder structure. For simplicity, we'll use a basic version of Seq2Seq without attention or other advanced techniques like Transformers. This example will demonstrate how to set up a basic machine translation model from scratch in PyTorch.

Steps to Build a Basic Google Translate-like Model in PyTorch

1. **Prepare the Dataset:** We'll use a parallel corpus (e.g., English-German or English-French translations).
2. **Preprocessing:** Tokenizing text and converting sentences into numerical representations.
3. **Model Setup:** Build an Encoder-Decoder architecture in PyTorch.
4. **Training:** Train the model on the dataset.
5. **Translation (Inference):** Translate a sentence using the trained model.

1. Dataset

For this demonstration, let's assume we're working with the Multi30k dataset (a small parallel dataset with English-German translations). You can also use the WMT dataset for larger, production-level tasks.

First, you would need to load and preprocess the dataset. In real-world scenarios, datasets are tokenized and preprocessed using libraries like torchtext or spaCy. Below is an example of how to load and preprocess the data with torchtext.

Python code:

```
import torch
import torchtext
from torchtext.data import Field, BucketIterator

# Define the fields
SRC = Field(tokenize="spacy", tokenizer_language="en_core_web_sm",
            init_token="<sos>", eos_token="<eos>", lower=True)
TRG = Field(tokenize="spacy", tokenizer_language="de_core_news_sm",
            init_token="<sos>", eos_token="<eos>", lower=True)

# Load the dataset (using Multi30k for demonstration)
train_data, valid_data, test_data =
torchtext.datasets.Multi30k.splits(exts=(".en", ".de"), fields=(SRC, TRG))
```

```

# Build vocab

SRC.build_vocab(train_data, min_freq=2)
TRG.build_vocab(train_data, min_freq=2)

# Create iterators

BATCH_SIZE = 32

train_iterator, valid_iterator, test_iterator = BucketIterator.splits(
    (train_data, valid_data, test_data), batch_size=BATCH_SIZE,
device=device
)

```

2. Model Components

The Encoder and Decoder will be built using LSTM (Long Short-Term Memory) units.

2.1 Encoder

The encoder processes the source sentence (English) and generates hidden states that will be passed to the decoder.

Python code:

```

class Encoder(nn.Module):

    def __init__(self, input_dim, emb_dim, hidden_dim, n_layers, dropout):
        super(Encoder, self).__init__()
        self.embedding = nn.Embedding(input_dim, emb_dim)
        self.rnn = nn.LSTM(emb_dim, hidden_dim, n_layers, dropout=dropout,
batch_first=True)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src):
        embedded = self.dropout(self.embedding(src))
        outputs, (hidden, cell) = self.rnn(embedded)
        return hidden, cell

```

- input_dim: Source vocabulary size.
- emb_dim: Embedding dimension for words.
- hidden_dim: Number of hidden units in the LSTM.
- n_layers: Number of LSTM layers.
- dropout: Dropout for regularization.

2.2 Decoder

The decoder generates the translated sentence (German) based on the context from the encoder's hidden states.

Python code:

```
class Decoder(nn.Module):
    def __init__(self, output_dim, emb_dim, hidden_dim, n_layers, dropout):
        super(Decoder, self).__init__()
        self.output_dim = output_dim
        self.embedding = nn.Embedding(output_dim, emb_dim)
        self.rnn = nn.LSTM(emb_dim, hidden_dim, n_layers, dropout=dropout,
batch_first=True)
        self.fc_out = nn.Linear(hidden_dim, output_dim)
        self.dropout = nn.Dropout(dropout)
    def forward(self, input, hidden, cell):
        input = input.unsqueeze(1) # Add batch dimension (for 1 timestep)
        embedded = self.dropout(self.embedding(input))
        output, (hidden, cell) = self.rnn(embedded, (hidden, cell))
        prediction = self.fc_out(output.squeeze(1))
        return prediction, hidden, cell
```

2.3 Seq2Seq Model

The Seq2Seq model uses both the Encoder and Decoder together. It receives the source sentence (e.g., in English) and generates the target sentence (e.g., in German).

Python code:

```
class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, device, trg_pad_idx, max_length=100):
        super(Seq2Seq, self).__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device
        self.trg_pad_idx = trg_pad_idx
        self.max_length = max_length
    def forward(self, src, trg, teacher_forcing_ratio=0.5):
        batch_size = src.shape[0]
        trg_len = trg.shape[1]
        trg_vocab_size = self.decoder.output_dim
```

```

    # Tensor to hold the decoder outputs
    outputs = torch.zeros(batch_size, trg_len,
trg_vocab_size).to(self.device)
    # Encoder forward pass
    hidden, cell = self.encoder(src)
    # First input to the decoder is the <sos> token (start-of-sequence
token)
    input = trg[:, 0]
    # Decoding
    for t in range(1, trg_len):
        output, hidden, cell = self.decoder(input, hidden, cell)
        outputs[:, t] = output
        # Get the highest predicted token for the next time step
        top1 = output.argmax(1)
        # Decide if we are going to use teacher forcing or not
        input = trg[:, t] if torch.rand(1).item() < teacher_forcing_ratio else
top1
    return outputs

```

3. Training the Model

Training the model involves the following steps:

- Set up the optimizer.
- Define a loss function (usually CrossEntropyLoss for classification tasks).
- Iterate over the dataset, performing forward and backward passes, and updating model weights.

Python code:

```

# Hyperparameters
INPUT_DIM = len(SRC.vocab)
OUTPUT_DIM = len(TRG.vocab)
EMBEDDING_DIM = 256
HIDDEN_DIM = 512
N_LAYERS = 2
DROPOUT = 0.5
TRG_PAD_IDX = TRG.vocab.stoi['<pad>']
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```
# Initialize the encoder, decoder, and seq2seq model
encoder = Encoder(INPUT_DIM, EMBEDDING_DIM, HIDDEN_DIM,
N_LAYERS, DROPOUT).to(device)
decoder = Decoder(OUTPUT_DIM, EMBEDDING_DIM, HIDDEN_DIM,
N_LAYERS, DROPOUT).to(device)
model = Seq2Seq(encoder, decoder, device, TRG_PAD_IDX).to(device)
# Optimizer and loss function
optimizer = torch.optim.Adam(model.parameters())
criterion = nn.CrossEntropyLoss(ignore_index=TRG_PAD_IDX)
# Training loop
n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    epoch_loss = 0
    for batch in train_iterator:
        src = batch.src.to(device)
        trg = batch.trg.to(device)
        optimizer.zero_grad()
        # Forward pass
        output = model(src, trg)
        # Flatten the output and target tensors
        output_dim = output.shape[-1]
        output = output.view(-1, output_dim)
        trg = trg.view(-1)
        # Compute the loss
        loss = criterion(output, trg)
        loss.backward()
        # Update the parameters
        optimizer.step()
        epoch_loss += loss.item()
    print(f'Epoch {epoch+1}, Loss: {epoch_loss / len(train_iterator)}')
```

4. Inference/Translation

For inference, we can use Greedy Search (picking the most likely word at each time

step) or Beam Search for more sophisticated decoding. Here's how we would translate a sentence using Greedy Search:

Python code:

```
def translate_sentence(sentence, model, src_vocab, trg_vocab, device,  
max_length=100):  
    model.eval()  
  
    # Tokenize and convert the sentence to indices  
    tokens = [token.lower() for token in sentence.split()]  
    src_indices = [src_vocab.stoi[token] for token in tokens]  
    src_tensor = torch.tensor(src_indices).unsqueeze(0).to(device)  
  
    # Get encoder hidden state  
    with torch.no_grad():  
        hidden, cell = model.encoder(src_tensor)  
  
    # Start with the <sos> token  
    trg_indices = [trg_vocab.stoi['<sos>']]  
  
    # Decode the sentence  
    for t in range(max_length):  
        trg_tensor = torch.tensor([trg_indices[-1]]).to(device)
```

Conclusion:

PyTorch is a versatile and powerful deep learning library that finds applications across many industries. Its ease of use, flexibility, and support for GPU acceleration make it a preferred choice for building state-of-the-art models in computer vision, NLP, reinforcement learning, healthcare, and many other fields. The community and ecosystem around PyTorch continue to grow, ensuring that its applications will only increase in scope and sophistication.

Questions:

- Q1. What are some commonly used datasets for machine translation, and how can they be loaded in PyTorch?
- Q2. What is the role of the encoder and decoder in the seq2seq architecture?
- Q3. What metrics can you use to evaluate a machine translation model? (e.g., BLEU score)